The Journal of Open Source Software

ParaDiGM: Parallel Distributed General Mesh

Eric Quémerais [1], **Bastien Andrieu** [1], **Bruno Maugars** [2], **Karmijn Hoogveld** [1], **Clément Benazet** [2], **Julien Coulet** [2], **Berenger Berthoul** [2], **Nicolas Dellinger** [3], **and Thomas Hennion** [1]

1 DMPE, ONERA, Université Paris-Saclay, 92320 Châtillon, France **2** DAAA, ONERA, Université Paris-Saclay, 92320 Châtillon, France **3** DMPE, ONERA, Université de Toulouse, 31000 Toulouse, France

DOI: 10.21105/joss.10483

Software

- Review
- Repository
- Archive

Editor: Christoph Junghans

Reviewers:

- @DamynChipman
- @jacobj-LANL

Submitted: 03 April 2026
Published: 25 October 2026

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

ParaDiGM (**Para**llel **Di**stributed **G**eneral **M**esh) is an open-source C library (LGPL) designed to overcome the technological bottlenecks associated with geometric data handling and mesh manipulation in massively parallel numerical simulations. Originally developed as the geometric engine for the *CWIPI* coupling library (Quémerais et al., 2026), it has evolved into a standalone infrastructure designed to manage distributed meshes at the billion-element scale and beyond.

Unlike traditional mesh infrastructures, ParaDiGM acts as a **non-intrusive progressive middleware**. It provides solvers (CFD, structural mechanics, high-order Discontinuous Galerkin) with a suite of low-level services for hybrid partitioning, point cloud location, and wall distance computation, all while ensuring the dynamic load balancing essential for **exascale** performance. With native APIs for **C/C++**, **Fortran**, and **Python/NumPy**, ParaDiGM seamlessly integrates into existing production codes—such as *CEDRE* (Refloch et al., 2011), *elsA* (Cambier et al., 2013), *SoNiCS* (Lienhardt & Michel, 2025) or *Maia* (Coulet, 2026) — transforming static data structures into dynamic and highly scalable geometric environments.

Statement of Need

ParaDiGM originated from the need to provide high-performance field projection capabilities in the context of multi-physics coupling, as required by the *CWIPI* coupling library (Quémerais et al., 2026). This initial use case — efficiently locating points across distributed meshes and interpolating physical fields between non-conforming discretizations — exposed the broader bottlenecks that any geometric middleware must address at scale.

In the era of exascale computing, numerical simulation faces a critical paradigm shift. In **Computational Fluid Dynamics (CFD)**, mesh management can no longer rely on centralized or semi-distributed approaches. Integrating distributed mesh capabilities into existing production solvers often encounters major architectural and performance barriers.

Geometric algorithms, once confined to pre-processing, are now required repeatedly within the solver's main execution loop. Modern studies involve evolving meshes, driven either by moving bodies or Dynamic Mesh Adaptation. Whether computing wall distances for turbulence modeling or performing mesh repartitioning, these operations must deliver execution times of the same order of magnitude as a solver iteration — failing to meet this constraint creates a performance bottleneck that severely penalizes overall simulation efficiency. A further challenge lies in data distribution: the initial partition provided to geometric algorithms is typically optimized for physical computation, which is often sub-optimal for geometric operations. Without internal dynamic load balancing, these operations create computation imbalances and memory bottlenecks, compromising simulation execution at very large scale.

State of the Field

Several parallel mesh management frameworks exist, each addressing different aspects of distributed geometric computing. General-purpose platforms such as *Trilinos* (Heroux et al., 2005) and *Arcane* (Grospellier & Lelandais, 2009) provide rich object hierarchies but impose significant architectural constraints on host solvers. Data exchange platforms such as *Salome* (MED format) (Ribes & Caremoli, 2007) rely on monolithic data models that introduce memory overhead incompatible with optimized solvers at scale. Mesh generation tools such as *Gmsh* (Geuzaine & Remacle, 2009) cover the CAD-to-mesh pipeline but do not address the in-solver geometric operations required by modern adaptive simulations.

At a lower level, *MOAB* (Tautges et al., 2004) shares several design principles with **ParaDiGM**: a Structure-of-Arrays (SOA) memory layout, and native APIs for C, Fortran, and Python/NumPy. **ParaDiGM** extends this philosophy by interpreting elements of arbitrary order regardless of their internal node numbering convention, and by leveraging element shape functions directly within geometric algorithms such as point cloud location — a capability not prominently exposed in *MOAB*'s core algorithms. The *PUMI* library (Ibanez et al., 2016), by contrast, provides a strongly C++-oriented API, which can limit its integration into legacy Fortran or C production solvers. **ParaDiGM** fills the remaining gap as a non-intrusive middleware: it operates on simple CSR arrays, integrates without refactoring the host solver, and delivers dynamically load-balanced geometric services compatible with solver iteration frequencies.

Software Design

ParaDiGM is a **Software Development Kit (SDK)** rather than a standalone application. Unlike tools like *Gmsh* (Geuzaine & Remacle, 2009), it is not intended for mesh generation from **CAD** models. Its role begins **as soon as a first mesh is obtained**: it provides solvers with the low-level functions needed to distribute, partition, and manipulate discretized meshes in a parallel fashion.

Figure 1 illustrates the various functionalities provided by ParaDiGM that can be leveraged by a numerical simulation software throughout its execution pipeline. Its philosophy is built on three pillars:

1. **Interoperability and Agnosticism:** Ensuring full compatibility with legacy languages (Fortran, C) and modern environments (Python/NumPy) for a wide variety of numerical methods (Finite Volumes, Finite Elements, high-order Discontinuous Galerkin).
2. **Hybrid Partitioning Strategy:** ParaDiGM unifies third-party solutions (e.g., *PT-Scotch* (Chevalier & Pellegrini, 2008), *ParMetis* (Karypis et al., 1997)) while supplementing them with native high-performance algorithms like **Space Filling Curves (SFC)** for frequent repartitioning.
3. **Total Distribution and Geometric Performance:** By ensuring homogeneous load distribution, the framework has enabled the development of highly efficient algorithms, such as **distributed point cloud location** (Andrieu et al., 2026), achieving performance levels compatible with solver iteration frequencies.

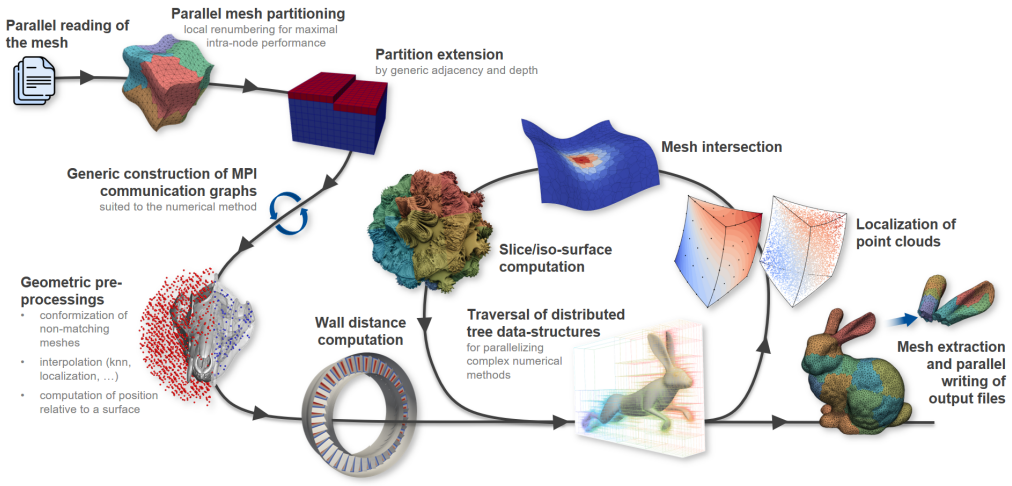


Figure 1: Example of a simulation chain using the features offered by ParaDiGM.

Research Impact Statement

As a high-performance middleware, ParaDiGM provides essential geometric services—such as parallel wall distance computation, point cloud location, and dynamic repartitioning—to various **numerical simulation and pre-processing software**, including *CEDRE* (Refloch et al., 2011), *elsA* (Cambier et al., 2013), *SoNiCS* (Lienhardt & Michel, 2025), *MoDeTheC* (Dellinger et al., 2024) and *Maia* (Coulet, 2026). By generalizing the **Partitioned** and **Distributed View** concepts, it ensures that every stage of the computation remains memory-balanced, even for meshes at the billion-element scale and beyond. With native APIs for **C/C++**, **Fortran**, and **Python/NumPy**, ParaDiGM serves as a versatile infrastructure for aerospace research and large-scale computational physics in the exascale era.

Conclusion

ParaDiGM provides a production-ready, non-intrusive middleware for distributed mesh management at exascale, with demonstrated integration in major aerospace solvers. The library is under active development: ongoing work focuses on porting its most critical components to GPU architectures (Cazalbou et al., 2024), with particular attention to hybrid parallel octree construction and traversal — the most memory- and time-intensive steps in geometric algorithms. By offloading these structures to GPUs, ParaDiGM aims to maintain the load balance and iteration-compatible turnaround times required for next-generation heterogeneous computing environments.

AI Usage Disclosure

Generative AI tools were used to assist with the writing of this paper: translation, formatting, and improving the fluency of the text. No AI was used in the development of the ParaDiGM software. All scientific content was written and validated by the authors.

Acknowledgements

The authors would like to thank the following people for their contributions to the development, testing, and dissemination of the ParaDiGM library within the community:

Sébastien Bourasseau[1], Nicolas Lantos[1], Niels Guilbert[1], Alain Hervault[1], Julien Magnenet[1], Lucas Manueco[1], Lionel Matuszewski[1], Yacine Mezemate[1], Bertrand Michel[1], Christina Paulin[1], Christophe Peyret[1], Julien Vanharen[1], Jérôme Esbrat[2], Victor Pacotte[2], Bruno Peres[2] and Mickael Philit[3].

[1] ONERA, [2] EOLEN, [3] Safran.

The authors also wish to thank BPI France, the Directorate General for Civil Aviation (DGAC), and the General Scientific Directorate of ONERA for their financial support.

Author Contributions

The contributions to this software are listed according to the CRediT taxonomy:

- **Eric Quémerais**: Conceptualization, Methodology, Software, Validation, Writing – Review & Editing, Project Administration, Funding Acquisition, Supervision.
- **Bastien Andrieu**: Software, Validation, Writing – Original Draft.
- **Bruno Maugars**: Software, Validation.
- **Karmijn Hoogveld**: Software, Validation, Writing – Original Draft.
- **Clément Benazet**: Software, Validation.
- **Julien Coulet**: Software, Validation.
- **Bérenger Berthoul**: Software.
- **Nicolas Dellinger**: Software, Validation.
- **Thomas Hennion**: Software, Validation.

References

Andrieu, B., Maugars, B., & Quémerais, E. (2026). Dynamic load-balanced point location algorithm for data mapping. *Comptes Rendus. Mécanique*, *354*(G1), 53–70. https://doi.org/10.5802/crmeca.335

Cambier, L., Heib, S., & Plot, S. (2013). The onera elsA CFD software: Input from research and feedback from industry. *Mechanics & Industry*, *14*, 159–174. https://doi.org/10.1051/meca/2013056

Cazalbou, R., Duchaine, F., Quémerais, E., Andrieu, B., Staffelbach, G., & Maugars, B. (2024). *Hybrid multi-GPU distributed octrees construction for massively parallel code coupling applications*. https://doi.org/10.1145/3659914.3659928

Chevalier, C., & Pellegrini, F. (2008). PT-scotch: A tool for efficient parallel graph ordering. *Parallel Computing*, *34*(6), 318–331. https://doi.org/10.1016/j.parco.2007.12.001

Coulet, J. (2026). *Maia*. https://github.com/onera/maia.

Dellinger, N., Leplat, G., Huchette, C., Biasi, V., & Feyel, F. (2024). Numerical modeling and experimental validation of heat and mass transfer within decomposing carbon fibers/epoxy resin composite laminates. *International Journal of Thermal Sciences*, *201*, 109040. https://doi.org/10.1016/j.ijthermalsci.2024.109040

Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A 3-d finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, *79*(11), 1309–1331. https://doi.org/10.1002/nme.2579

Grospellier, G., & Lelandais, B. (2009). *The arcane development framework*. https://doi.org/10.1145/1595655.1595659

Heroux, M., Bartlett, V., R.and Howle, Hoekstra, R., Hu, J., Kolda, T., Lehoucq, R., Long, K., Pawlowski, R., Phipps, E., & others. (2005). An overview of the trilinos

project. *ACM Transactions on Mathematical Software (TOMS)*, *31*(3), 397–423. https://doi.org/10.1145/1089014.1089021

Ibanez, D. A., Seol, E. S., Smith, C. W., & Shephard, M. S. (2016). PUMI: Parallel unstructured mesh infrastructure. *ACM Trans. Math. Softw.*, *42*(3). https://doi.org/10.1145/2814935

Karypis, G., Schloegel, K., & Kumar, V. (1997). *PARMETIS: Parallel graph partitioning and sparse matrix ordering library* (No. 97-060). University of Minnesota. https://hdl.handle.net/11299/215345

Lienhardt, M., & Michel, B. (2025). SoNICS: Design and workflow of a new CFD software. *Comptes Rendus. Mécanique*, *353*, 1261–1287. https://doi.org/10.5802/crmeca.334

Quémerais, E., Andrieu, B., & Hoogveld, K. (2026). CWIPI: Coupling With Interpolation Parallel Interface. *Journal of Open Source Software*, *11*, 10424. https://doi.org/10.21105/joss.10424

Refloch, A., Courbet, B., Murrone, A., Villedieu, P., Laurent, C., Gilbank, P., Troyes, J., Tessé, L., Chaineray, G., Dargaud, J. B., Quémerais, E., & Vuillot, F. (2011). CEDRE Software. *Aerospace Lab*, *2*, p. 1–10. https://hal.science/hal-01182463

Ribes, A., & Caremoli, C. (2007). Salomé platform component model for numerical simulation. *31st Annual International Computer Software and Applications Conference (COMPSAC 2007)*, *2*, 553–564. https://doi.org/10.1109/COMPSAC.2007.185

Tautges, T. J., Ernst, C., Stimpson, C., Meyers, R. J., & Merkley, K. (2004). *MOAB : A mesh-oriented database*. Sandia National Laboratories. https://doi.org/10.2172/970174